

DocDrift

Documentation that updates itself when the code changes

Theme: 06 — Production · Function: AI-Powered Production

Suggested stack: LLM (any) · AST parsing · Embeddings · React · Optional MS: Azure OpenAI · Azure Functions · Cosmos DB · GitHub App

Problem Statement

Problem Background

Technical documentation — READMEs, API references, architecture notes, runbooks — goes stale the moment code changes. Under delivery pressure nobody updates it, so consumers trust docs that are silently wrong. There is no system that knows when a specific code change has invalidated a specific piece of documentation; doc rot is invisible until it costs someone hours.

Why It Matters

Stale docs cause integration bugs, slow onboarding, repeated questions to senior engineers, and broken handoffs. Documentation is treated as a manual afterthought rather than a living artifact the system maintains — which is exactly the kind of legacy assumption an AI-first production function should overturn.

Solution Summary

Why This Problem Was Chosen

In an AI-first production world, documentation should be maintained by the system, not by whoever remembers. Linking docs to the code they describe makes drift measurable and fixable automatically — a small, high-leverage change to the toolchain.

Proposed Solution

DocDrift links each documentation block to the code symbols it describes (functions, endpoints, config keys, schemas). On every commit or PR it diffs the changed symbols, finds the docs that reference them, scores how far each doc has drifted, and an LLM drafts the corrected text as a suggested documentation PR. A Doc Health dashboard shows a freshness score per repo and surfaces the most stale, most-read docs first.

Expected Impact

- Cut stale-documentation incidents by detecting drift the moment code changes — not months later.
- Faster onboarding and fewer repeat questions to senior engineers.
- Doc updates drafted in seconds as a reviewable PR rather than a forgotten chore.
- A measurable Doc Health score that makes documentation quality visible to the team.

Technical Approach & Implementation

Solution Workflow

1. A commit or PR is opened. DocDrift parses the diff and extracts the changed code symbols (AST + diff).
2. Each changed symbol is matched to the documentation blocks that reference it via a symbol-to-doc index built from embeddings + explicit links.
3. A drift score is computed per doc block from the number and recency of changed symbols it depends on.
4. For drifted blocks, an LLM rewrites the affected text using the diff as context, preserving voice and formatting.
5. Updated text is opened as a documentation PR / inline suggestion for human approval.
6. The Doc Health dashboard updates the repo freshness score and the prioritised stale-doc list.

Key Features

Symbol-to-Doc Mapping. Automatically links every doc block to the code symbols it describes, so the system knows what each sentence depends on.

Drift Detection & Scoring. Quantifies how stale each doc block is based on the code changes affecting it, weighted by recency.

LLM Rewrite Suggestions. Drafts corrected documentation as a reviewable PR, preserving the original tone and structure.

Doc Health Dashboard. Repo-level freshness score plus a prioritised queue of the most stale, most-read documents.

Technology Stack

Frontend

- React + TypeScript dashboard
- Recharts for the Doc Health score
- Markdown diff viewer

Backend

- Python 3.11 + FastAPI (webhook receiver)
- Background worker for analysis
- Queue for commit events

AI / ML

- LLM for doc rewriting (provider-agnostic)
- Embeddings to link prose to code symbols
- tree-sitter for AST symbol extraction

Data & Integrations

- Vector index for symbol-to-doc mapping
- Git provider API (GitHub / Azure DevOps) for diffs and PRs
- Doc store (Markdown in-repo)

Models & Algorithms

Symbol Extraction. tree-sitter parses changed files into an AST and extracts the touched symbols (functions, endpoints, config keys).

Doc-to-Symbol Linking. Embedding cosine similarity links doc paragraphs to the code symbols they describe, refined by explicit references in the text.

Drift Score. Weighted blend of (changed symbols a doc depends on) × (recency), normalised per repo to a 0–100 staleness score.

Doc Rewrite. LLM receives the old doc block + the symbol diff and produces a minimal corrected version, kept consistent in tone.

Innovation

Docs as linked, monitored artifacts. Every documentation block is tied to the code it describes, so drift becomes a measurable signal rather than a guess.

Drift score, not all-or-nothing. The system pinpoints exactly which sentences are stale instead of marking whole files out of date.

Auto-drafted update PRs. Engineers approve a correction in seconds instead of writing it from scratch under pressure.

Future Scope

Near-term

- API contract docs generated directly from endpoint signatures
- Slack / Teams nudge when a high-traffic doc drifts
- Per-author doc-debt report for engineering leads

Medium-term

- Executable doc examples — verify that code snippets in docs still run
- Multilingual doc sync from a single source of truth
- Auto-generated changelog / release notes from merged diffs

Long-term

- Org-wide knowledge graph linking docs, code, and people
- Confidence-gated auto-merge for trivial doc corrections
- Doc quality benchmarking across teams

Scalability & Larger Vision

How It Scales

DocDrift is built to grow along three independent axes without re-architecting the core.

Technically, the pipeline is event-driven and stateless: webhooks feed a commit queue, and background workers process diffs in parallel. Adding capacity is a matter of adding workers, so throughput scales linearly with load rather than degrading as repositories grow busier. The symbol-to-doc index lives in a vector store that comfortably holds millions of symbols, and

because symbol extraction runs through tree-sitter, support for a new programming language is a grammar plug-in, not a rewrite. The LLM layer is provider-agnostic, so the same system runs on a hosted model, a private endpoint, or a self-hosted model depending on the customer's data-residency needs.

Across repositories and teams, DocDrift moves naturally from a single repo to an entire organisation. The Doc Health score is defined per repo but aggregates cleanly to team, department, and org level, so the same primitive that helps one squad becomes a leadership-level quality signal without new machinery.

Organisationally, the unit of value — a linked, monitored doc block — is identical whether a team has ten documents or ten thousand. Onboarding a new repo requires no manual mapping; the embedding-based linker bootstraps the symbol-to-doc index automatically on first scan.

How It Expands

The roadmap deepens the same core idea rather than bolting on unrelated features. Near term, contract docs are generated directly from endpoint signatures and high-traffic drift triggers a Slack or Teams nudge. In the medium term, executable doc examples verify that code snippets in the docs still run, and a single source of truth syncs across languages. Long term, DocDrift becomes the backbone of an org-wide knowledge graph linking docs, code, and the people who own them, with confidence-gated auto-merge handling trivial corrections without human review.

The Larger Vision

Documentation stops being a manual afterthought and becomes a living layer the system maintains. The end state is an organisation where docs are structurally incapable of silently lying — every sentence is tied to the code it describes, and drift is a measurable, actionable signal rather than a surprise discovered hours into a debugging session.

Potential Impact

At one team's scale, DocDrift saves hours per stale-doc incident and accelerates onboarding. At org scale, the compounding effect is larger: a measurable Doc Health benchmark across teams, sharply fewer repeat questions to senior engineers, and documentation that becomes a trusted asset rather than a known liability. The intervention is small and high-leverage — a change to the toolchain — but it shifts an entire engineering culture from “docs rot” to “docs self-heal.”

SprintSense

Turn a messy backlog into a realistic, risk-aware sprint plan

Idea 02 — AI-Augmented Project Management · Theme: 06 — Production · Function: AI-Powered Production
Suggested stack: LLM (any) · Embeddings · Lightweight regression · React · Optional MS: Azure OpenAI · Azure DevOps Boards / GitHub Issues · Power BI

Problem Statement

Problem Background

Sprint planning is largely guesswork. Estimates are inconsistent between people, implicit dependencies between tickets are missed, and teams routinely over-commit. Slippage is discovered at the end of the sprint during the retro — not predicted at the start when something could still be done about it. Project-management tools track work but do not reason about it.

Why It Matters

Missed sprints erode trust with stakeholders, break roadmaps, and force crunch. Even modest gains in estimate accuracy and an early warning when a sprint is going off track compound into far more predictable delivery — one of the clearest wins available to the production function.

Solution Summary

Why This Problem Was Chosen

AI-augmented project management is called out directly in the theme. Estimation, dependency reasoning, and slippage forecasting are exactly the kind of pattern-over-history tasks an LLM plus a team's own data can do well, and the result is immediately useful to every team.

Proposed Solution

SprintSense ingests the backlog (tickets, descriptions, labels) together with the team's historical velocity and cycle-time data. It estimates each item from its text and similar past items, detects implicit dependencies between tickets, and proposes a capacity-aware sprint plan that actually fits the team. Once the sprint starts, it produces a daily slippage forecast that updates as work moves — flagging the specific at-risk items, with reasons, while there is still time to react.

Expected Impact

- More accurate, consistent estimates grounded in the team's own delivery history.
- Fewer over-commitments through capacity-aware planning.
- Slippage flagged mid-sprint with the specific items at risk — not at the retro.
- Less crunch and more predictable roadmaps for stakeholders.

Technical Approach & Implementation

Solution Workflow

1. Pull the backlog from the issue tracker (descriptions, labels, links).

2. An LLM estimates each item using its text plus the most similar previously-completed items.
3. Dependency detection infers implicit ordering between tickets from their content and explicit links.
4. A capacity-aware scheduler assembles a candidate sprint that respects team capacity and dependency order.
5. During the sprint, actual progress is compared against the forecast each day.
6. A slippage alert fires with the at-risk items and the reason, while there is still room to rebalance.

Key Features

History-Grounded Estimation. Estimates each ticket from its text and the team's own similar past items, not a generic model.

Dependency Graph Detection. Surfaces implicit dependencies between tickets so the plan is sequenced correctly.

Capacity-Aware Auto-Plan. Builds a candidate sprint that fits real team capacity and dependency order.

Live Slippage Forecast. Daily probability the sprint completes, with the specific items dragging it off track.

Technology Stack

Frontend

- React + TypeScript
- Gantt + burndown charts
- Drag-to-adjust sprint board

Backend

- Python 3.11 + FastAPI
- Scheduler service (capacity bin-packing)
- Nightly forecast job

AI / ML

- LLM for estimation + dependency reasoning
- Embeddings for nearest-neighbour over past tickets
- Lightweight regression on cycle time

Data & Integrations

- Issue tracker API (Azure DevOps Boards / GitHub Issues / Jira)
- Historical velocity store
- Monte-Carlo forecast engine

Models & Algorithms

Estimation. Embedding nearest-neighbour over completed tickets gives a baseline; an LLM adjusts for specifics in the description.

Dependency Detection. LLM classifies whether ticket A blocks ticket B, combined with explicit tracker links, to build a DAG.

Scheduler. Capacity bin-packing that respects the dependency DAG and per-member availability.

Slippage Forecast. Monte-Carlo simulation over remaining work versus the team's velocity distribution, run daily.

Innovation

Estimates from your own history. The model learns this team's pace from completed work, instead of imposing a one-size-fits-all velocity.

Predictive, not retrospective. Slippage is forecast on day two, not explained at the retro — turning planning into a live control loop.

Plan + forecast in one loop. The same engine that builds the plan continuously checks whether reality is keeping up with it.

Future Scope

Near-term

- Cross-team dependency view for shared roadmaps
- Daily standup digest auto-generated from board movement
- What-if scope simulator ("if we drop X, do we finish?")

Medium-term

- Automatic mid-sprint rebalancing suggestions
- Risk-aware roadmap planning across multiple sprints
- Estimate-accuracy feedback loop per team and per author

Long-term

- Portfolio-level delivery forecasting across many teams
- Capacity planning tied to hiring and leave calendars
- Predictive staffing recommendations for upcoming epics

Scalability & Larger Vision

How It Scales

SprintSense is designed to scale from a single team's board to portfolio-level delivery forecasting on the same engine.

Technically, ingestion runs through pluggable adapters, so adding a new issue tracker (Azure DevOps Boards, GitHub Issues, Jira) is a connector, not a redesign. Estimation relies on embedding nearest-neighbour search over completed tickets, which scales to large backlog

histories, and the Monte-Carlo slippage forecast is embarrassingly parallel — more teams simply mean more independent simulation jobs.

Across teams, each team gets its own history-grounded model, so accuracy improves with the team's own data instead of being diluted by a one-size-fits-all velocity. Because every team is modelled independently, the system scales horizontally: a hundred teams are a hundred parallel forecasts, not a single bottlenecked model.

Organisationally, the same primitives — estimate, dependency graph, capacity-aware plan, live forecast — roll up from a single sprint to cross-team roadmaps to a portfolio view, giving leadership an aggregated delivery forecast built from the same trusted signals each team already sees.

How It Expands

Expansion turns planning into an increasingly autonomous control loop. Near term, a cross-team dependency view and a what-if scope simulator help teams reason about trade-offs live. In the medium term, the system suggests automatic mid-sprint rebalancing and supports risk-aware planning across multiple sprints. Long term, capacity planning ties directly into hiring and leave calendars, and SprintSense produces predictive staffing recommendations for upcoming epics — connecting delivery forecasting to the resourcing decisions that drive it.

The Larger Vision

Project management shifts from tracking work to reasoning about it. Slippage is forecast on day two and corrected mid-flight, not explained at the retro. At full scale, an organisation gains a continuously updating, bottom-up forecast of whether its roadmap is achievable — turning planning from an annual act of optimism into a live, data-grounded control system.

Potential Impact

For a single team, SprintSense delivers more accurate estimates, fewer over-commitments, and early warnings while there is still time to react. Aggregated across an organisation, modest per-team gains in estimate accuracy compound into materially more predictable delivery: rebuilt stakeholder trust, roadmaps that hold, and far less crunch. It is one of the clearest, most repeatable wins available to the production function — and it gets more valuable the more teams adopt it, because every completed sprint sharpens the model.